Engineering Retrospective

Campus Ride-Pooling and Split-Fare System

Compiled from the project's records, handout section 19

Prithviraj Ghosh (230794) Mainak Sarkar (230619) Khushi Ranawat (230562) Mahir Jain (230618) Chaitanya Nitawe (230315)

1. What went wrong during development?

- **Testing started late.** `master` has no commits for 12 days between 15 Oct and 28 Oct, and Deliverable 3 testing started only at the end of that gap.
- **Tests stopped at the mocks.** The unit suite cleared its 80 % coverage gate with the database mocked, so defects that only appear on a real database stayed hidden until the integration suite ran.
- **Deployment came last.** The platform was chosen and deployed on 5 November, the day before the deadline, which left little time to run the system under real conditions.
- **Jira lagged behind the work.** 49 issues were marked resolved on 5 November, and 24 of the 26 defect Bugs were recorded after their fixes.
- **Review was not enforced.** 68 of the 98 merged pull requests have no recorded review from another member, because `master` has no branch protection.

2. Which requirements changed and why?

The deployment platform changed from GCP (requirement NFR-RD-18) to Render.com on 5 November. Render builds the repository's Docker image directly from GitHub, provides managed PostgreSQL and runs the custom server that Socket.IO needs. The submitted requirement still says GCP, and section 7 of the deployment documentation records the change. The D1 review also found ten product defects against the requirements (#67). Those changed the code to match the requirements, not the requirements themselves.

3. Which architecture or design decision would you change?

- **Build on the final platform from the start.** A walking skeleton on the real host during Deliverable 2 would have tested the deployment choice while there was still time to change it.
- **Design ride search against its latency target.** A SQL optimisation lowered p95 latency from 2.27 s to 1.76 s at 100 virtual users, but the 750 ms target was not met. Treating the target as a design input, not a test at the end, would have shaped the query earlier.
- **Plan for more than one server process.** Socket.IO runs inside the single custom Node.js server, so running a second instance would need a shared adapter that the design does not have.

4. What did the LLM get wrong?

- **A weak safety check.** The first integration harness guarded its database reset by name alone, which could still have wiped an existing database with a matching name. Khushi's review caught it (#83).
- **A wrong citation that spread.** A review draft cited #94 for two defects whose code fixes are in #91. This reached Jira and the bug report before it was corrected.
- **Errors in review drafts.** One defect's before and after behaviour was the wrong way round, a draft named a route that does not exist, and a count was off by one. All were caught before the reviews were shared.
- **A chart that looked right in source.** In the first Mermaid timeline, 9 of 15 task bars had zero width. Only rendering it showed this (#99).
- **Facts in report drafts.** A draft said five threats where the security report has six, and said every AI feature falls back to deterministic behaviour, which is not true of all three.

5. Where did AI substantially improve productivity?

- **Test suites.** The integration harness and over 100 integration and adversarial tests were written across two days, 28 and 29 October, and they found real defects (#84, #87).

- **Generated evidence.** The traceability matrix and the report's GitHub and Jira numbers are built by scripts from recorded data, which would have been slow and error-prone by hand.
- **Bulk project work.** The final Jira cleanup moved 42 issues and assigned 26 Bugs through a connector, with each change checked by a follow-up search.
- **Overall.** The size-based estimate in section 9 of the costing report is about three times the member-days with a commit. Commit days undercount effort, so this suggests a gain without measuring it.

6. Where did AI introduce risk or additional work?

- **Confident errors.** Wrong statements read exactly like right ones, so every claim needed checking against the source. The #91 and #94 error shows the cost when a check is missed.
- **Volume outpacing review.** Generated code, tests and reports arrive faster than people can read them, and the review record above shows that review did not keep up.
- **Green checks that prove little.** Passing tests and high coverage from generated tests over mocks looked like assurance, and the integration suite showed the limit of that.

7. What was the most difficult bug or failure?

Every seat request and the payment reminder scheduler failed with a 500 on a real database. A JavaScript `Date` used in `$n - interval` reached PostgreSQL untyped, and the mocked unit tests could not see it. The integration suite exposed it, and #84 casts the parameter to `timestamptz`, with regression tests. A related test then failed intermittently, because seeded rows took their time from the database and the code from Node.js, about 1 ms apart. The team traced that to its cause rather than retrying the test.

8. How did Jira and GitHub support the SDLC?

Jira gave the work a backlog, sprints and a Bug record, and GitHub gave it pull requests, reviews and three CI jobs on every pull request. Together they support the traceability matrix, which follows a requirement through its Jira issue and pull request to its test: 77 of the 98 merged pull requests name a Jira key. They worked best where review actually happened, as in #83. They were weakest as a planning record: 3 of 173 issues have an estimate and 28 are in no sprint.

9. What would you do differently in a real production project?

- Require review through branch protection from the first day, and update Jira as the work happens.
- Deploy first, and run integration tests against a real database from the first feature.
- Use a paid database with backups: the Free tier has none, and this one expires on 5 December 2026.
- Add monitoring, alerting and secret scanning before launch, and meet latency targets before release.
- Estimate before the work and track time, so that estimates can be compared with something better than commits.